第7回：DB・APIを使うLLM

Function Callingで、決められた関数を呼ぶ

今日のゴールと流れ

LLMの提案と、Pythonによる実行を区別する。

時間	内容
10分	文書検索とDB取得の違い
20分	関数定義と引数の形式
20分	呼び出し結果の受け渡し
30分	SQL実装と検証
10分	制限・振り返り

文書検索と構造化データ

質問	参照先
SQLiteとは何か	学習ノート
課題を提出する方法	案内文書
未提出の課題は何件か	課題DBのstatus

更新される状態は、DBの条件検索で取得すると扱いやすくなります。
この教材の課題DBは架空データで、実在の学生の情報ではありません。

Function Callingとは

1. アプリが使える関数の名前と引数をLLMに伝える
2. LLMが関数名と引数を提案する
3. Pythonが提案を検証して関数を実行する
4. 実行結果をLLMへ返す
5. LLMが結果を使って文章を作る

LLMが直接SQLiteを操作するわけではありません。

1問を追うための役割分担

質問：「未提出の課題を教えて」

担当	この例で行うこと
LLM・1回目	get_assignments と pending を提案
Python	関数名と引数を確認してSQLを実行
SQLite	条件に一致する課題を返す
LLM・2回目	取得した課題を使って回答文を生成

以下は処理を追う説明例です。モデルの実際の出力は実行して確認します。

今回使える関数

```
get_assignments(status)
```

status	取得するもの
all	すべての課題
pending	未提出の課題
done	提出済みの課題

読み取りだけです。削除や提出済みへの変更は許可しません。

初期状態の課題DB

id	title	status
1	第2回 API呼び出しレポート	done
2	第5回 検索結果の比較	pending
3	第8回 評価シート	pending

教材の架空データです。未提出は2件、提出済みは1件です。
既存の課題の状態は、初期化や再起動で上書きしません。

関数の説明を渡す

```
{
    'type': 'function',
    'name': 'get_assignments',
    'description': '教材用の架空の課題の提出状況を調べる。',
    'parameters': {...},
    'strict': True,
}
```

説明文は、LLMが用途を判断する材料です。

`parameters` に、受け付ける引数の形を記述します。

引数の形

```
'properties': {
    'status': {
        'type': 'string',
        'enum': ['all', 'pending', 'done']
    }
},
'required': ['status'],
'additionalProperties': False
```

選択肢と必須項目を指定します。サーバー側でも再度検証します。

自然文と引数の対応

利用者の質問	期待するstatus
未提出の課題を教えて	pending
提出済みの課題は？	done
すべての課題を一覧にして	all

日本語の質問を、関数が受け取れる選択肢へ対応させます。
期待する引数と、実際にモデルが提案した引数を比較します。

最初のAPI呼び出し

```
first = client.responses.create(
    model=MODEL, reasoning={'effort': 'none'},
    instructions=instructions,
    input=history, tools=TOOLS,
    parallel_tool_calls=False,
    max_output_tokens=600, store=False,
)
```

関数の情報を `tools` に入れます。まだ関数は実行していません。

関数呼び出しの読み取り

```
for item in first.output:
    if item.type == 'function_call':
        rows = execute_tool(item.name, item.arguments)
```

出力には文章など、関数以外の項目もあります。

`arguments` はJSON文字列なので、`json.loads()` で辞書へ変換します。

1回目の出力の例

主要な項目だけを示した、説明用の出力です。

```
{
  "type": "function_call",
  "name": "get_assignments",
  "arguments": "{\"status\":\"pending\"}",
  "call_id": "call_example_01"
}
```

この時点ではDBをまだ検索していません。

`arguments` の中身はJSON文字列なので、Pythonで解析します。

実行前の検証

```
if name != 'get_assignments':
    raise ValueError('許可していない関数です。')
args = json.loads(arguments)
```

続けて、辞書か、キーが `status` だけか、値が許可した文字列かを調べます。
モデルが作った値も、外部からの入力として扱います。

検証を通る引数・止まる引数

関数名が `get_assignments` の場合です。

argumentsの内容	結果
{"status":"pending"}	許可された文字列なので実行
{"status":"unknown"}	選択肢にないので拒否
{"status":1}	文字列ではないので拒否
{"status":"all","delete":true}	余分なキーがあるので拒否

`strict` の指定に加えて、実行するPython側でも検証します。

SQLiteの条件検索

```
cursor.execute(
    'SELECT id, title, status FROM assignments WHERE status = ?',
    (status,),
)
```

SQLの形はPython側が決めます。

`?` に値を別の引数で渡す書き方は、前のWeb授業と同じです。

pendingの検索結果

初期DBで `get_assignments('pending')` を実行した結果です。

```
[
  {'id': 2, 'title': '第5回 検索結果の比較',
   'status': 'pending'},
  {'id': 3, 'title': '第8回 評価シート',
   'status': 'pending'}
]
```

返るのは辞書のリストです。この時点で件数と課題名を確認できます。
LLMを使う前に、関数単体でも同じ結果になるか確かめます。

実習1：SQLの穴埋め

```
cd session07/exercise
python init_db.py
```

`assignment_tools.py` の `get_assignments()` を完成させます。
`all` の場合と、それ以外の場合のSQLを書き分けます。
仮の `raise` を削除します。

実習2：関数だけを確認

ターミナルで `python` を起動します。

```
from assignment_tools import get_assignments
print(get_assignments('pending'))
print(get_assignments('done'))
print(get_assignments('all'))
```

モデルを使う前に、普通のPython関数として正しく動くか確認します。

結果を次の入力へ追加

```
history.extend(first.output)
history.append({
    'type': 'function_call_output',
    'call_id': item.call_id,
    'output': json.dumps(rows, ensure_ascii=False),
})
```

`call_id` で、どの呼び出しへの結果かを対応させます。
関数を提案した出力も、次の入力に引き継ぎます。

call_idで対応させるもの

1回目の出力	Pythonが追加する結果
type: function_call	type: function_call_output
call_id: call_example_01	call_id: call_example_01
arguments: pendingの指定	output: 取得した2件のJSON文字列

同じ `call_id` で、どの依頼に対する結果かを示します。
呼び出しの出力と、その結果の両方を `history` に入れます。
IDは説明用です。実装はモデルが返した `item.call_id` を使います。

2回目のAPI呼び出し

関数の結果を含む `history` を渡して、回答を生成します。
教材では2回目に `tools` を渡しません。

- 追加の関数呼び出しを許可しない
- 無限に処理を繰り返さない
- 2回分の入力・出力トークンを合計する

関数を呼ばずに回答する場合の分岐も、コードで確認します。

最終回答とDB結果の照合

説明用の回答例：

> 未提出は2件です。「第5回 検索結果の比較」と
> 「第8回 評価シート」です。

照合するのは、件数・課題名・提出状況です。
取得したデータにない締切や成績を、回答に足していないかも確認します。

関数を正しく呼べても、回答の確認は別に必要です。

実習3：自然文から課題を取得

`python main.py` で起動し、「課題DB」を選びます。
APIモードで、次の質問を試します。

- 「未提出の課題を教えて」
- 「提出済みの課題は？」
- 「すべての課題を一覧にして」

学習用の詳細欄で、関数名・引数・DBの結果を確認します。
デモは固定でpendingを実行するので、関数選択の評価には使えません。

失敗した段階の確認

観察した問題	確認するもの
pendingのはずがdoneになった	質問・関数の説明・実際の引数
pendingで提出済みの行が返る	Python関数とSQLの条件
DB結果は正しいが回答が違う	2回目の入力と生成結果
どの質問でもpendingになる	デモモードではないか

学習用の詳細欄で、引数とDB結果を回答と分けて確認します。

実習4：許可しない引数

```
from assignment_tools import execute_tool
execute_tool('delete_all', '{}')
execute_tool('get_assignments', '{"status":"unknown"}')
```

どちらも `ValueError` で止まることを確認します。
「全部削除して」という自然文でも、削除する関数自体がありません。

権限をコードで制限

- 関数名を固定する
- 値の選択肢を検証する
- 読み取りだけのSQLにする
- `eval()` 、 `exec()` 、モデルが生成したSQLを実行しない

プロンプトに「安全に」と書くだけでなく、
アプリが実行できる操作そのものを狭めます。

文書検索との組み合わせ

今回は画面で「資料検索」か「課題DB」かを選びます。
LLMによる自動的な使い分けは、発展課題です。

制作では、すべての機能を使う必要はありません。
自分の題材に合う取得方法を選び、その理由を説明します。

確認問題

1. 関数を実際に実行するのは、LLMとPythonのどちら？
2. 引数の形式をtoolsで指定したら、Pythonの検証は不要？
3. `call_id` は何を対応させる？
4. APIの利用量は1回目だけを数えればよい？

提出物と次回

提出：`assignment_tools.py` と `worksheet.md` のGitHub URL。

- pending・done・allの取得結果
- 許可していない関数や引数で止まること
- 関数選択と実行の役割の説明

次回は、完成したアプリを評価し、安全な運用を点検します。